

Seminar 4 – Backtracking în Prolog

- Prolog folosește backtracking pentru a găsi toate soluțiile. Până acum, am rezolvat probleme care aveau o singură soluție (de ex., să găsim maximul unei liste) și trebuia fie să adăugăm în clauze tăieturi care să oprească *backtracking*-ul, fie să adăugăm condiții pentru a ne asigura că avem o singură soluție.
- La acest seminar abordăm o problemă care are mai multe soluții, faptul că Prolog folosește backtracking devenind de mare ajutor. Vom implementa predicate care au mai multe soluții, acestea numindu-se, prin urmare, **predicate nedeterminate**.

1. Se dă șirul $a_1 \dots a_n$ format din numere întregi distincte.

Se cere să se genereze lista tuturor subșirurilor cu aspect de "vale" (o secvență are aspect de vale dacă elementele descresc până la un moment dat, iar apoi cresc).

- O primă variantă – foarte inefficientă – ar fi să rezolvăm problema folosind 2 predicate: unul care generează toate subșirurile (aranjamente de cel puțin 3 elemente) și altul care verifică dacă un subșir are sau nu aspect de vale. Problema este că un șir are foarte multe subșiruri diferite (mai ales că în cazul nostru ordinea elementelor în subșir contează) și este probabil ca o mare parte dintre ele să nu aibă aspect de vale.
De exemplu, să considerăm șirul [5,4,3,6,7,-1,-2,-3]. Câteva exemple de subșiruri cu aspect de vale sunt: [5,4,6], [6,4,5], [5,4,3,6,7], [-3,-2,-1,3,4,5], [-1,-3,-2], etc.
Dacă avem subșirul [5,3,7] (care este o vale) la această subșir nu mai putem adăuga elemente din lista originală (4,-1,-2,-3) deci nu e nevoie să mai continuăm să generăm subșiruri adăugând aceste elemente.
- De aceea, pentru a avea o soluție mai eficientă, va trebui să combinăm generarea subșirurilor cu verificarea condiției. Vom folosi o listă candidat, care reprezintă o soluție candidat, o soluție parțială. Vom adăuga elemente pe rând în această listă candidat, dar numai elemente cu care putem completa soluția parțială a.î. să se poată obține o soluție finală (o listă cu aspect de vale).
- Pentru a genera o listă cu aspect de vale avem nevoie de o secvență de elemente descrescătoare urmată de o secvență de elemente crescătoare. Când adăugăm elemente în lista candidat va trebui să ținem cont dacă suntem pe secvența de elemente descrescătoare sau dacă suntem pe secvența de elemente crescătoare. Pentru acest lucru vom reține un parametru în plus, care ne indică direcția pe care suntem. De exemplu:
 - o Valoarea 0 indică că suntem pe partea descrescătoare
 - o Valoare 1 indică că suntem pe partea crescătoare
- Lista candidat este o listă colectoare. Pentru a evita adăugarea la finalul listei colectoare (pe considerentul eficienței), vom opta să adăugăm elemente la începutul listei candidat. Dar aceasta presupune să generăm soluția în ordinea inversă (adică de la dreapta la stânga). De exemplu, soluția [5,4,3,6,7] va fi generată astfel:

- [7]
- [6,7]
- [3,6,7]
- [4,3,6,7]
- [5,4,3,6,7]
- Când adăugăm un element nou în lista candidat va trebui să ținem cont de 2 valori - direcția și primul element din lista candidat (care este practic ultimul element adăugat în lista candidat) - astfel:
 - Direcție 0 și element de adăugat mai mare decât primul element din candidat => mergem mai departe cu partea descrescătoare (ex. 8, [7,5,6], candidatul devine [8,7,5,6])
 - Direcție 1 și element de adăugat mai mic decât primul element din candidat => mergem mai departe cu partea crescătoare (ex. 5, [7,8], candidatul devine [5,7,8])
 - Direcția 1 și element de adăugat mai mare decât primul element din candidat => începem să generăm partea descrescătoare (ex. 9, [5,7,8], candidatul devine [9,5,7,8]).
 - Direcția 0 și element de adăugat mai mic decât primul element din candidat => nu putem adăuga (ex. 4, [7,6,5,8]).
- Când este o listă candidat o soluție? Pentru a avea o listă cu aspect de vale, ne trebuie atât secvență descrescătoare, cât și secvență crescătoare. Dintre aceste secvențe, prima dată se generează cea crescătoare, deci dacă lista candidat are o secvență descrescătoare (adică parametrul pentru direcție a devenit 0), avem o soluție. Dar, în același timp, este posibil ca această listă candidat să fie extinsă pentru a genera alte soluții.
- Cum generăm elementele care vor fi adăugate în lista candidat? Ne trebuie un predicat care să ne dea un element din lista inițială, care eventual este adăugat în candidat (în funcție de condițiile discutate mai sus), după care să ne dea un alt element ș.a.m.d.. Este primul predicat nedeterminist de care avem nevoie, unul care ne furnizează pe rând toate elementele dintr-o listă.

$$candidat(l_1 l_2 \dots l_n) = \begin{matrix} 1. l_1, n > 0 \\ 2. candidat(l_2 \dots l_n), n > 0 \end{matrix}$$

```
% candidat(L:list, E: element)
% model de flux: (i,o) nedeterminist, (i,i) determinist
% L - lista inițială, din care se generează elemente candidat
% E - un element din lista L
candidat([H|_], H).
candidat([_|T], E):-
    candidat(T, E).
```

- După ce am generat un element din listă, trebuie să verificăm ca acest element să nu mai apară în lista candidat. Pentru această verificare putem folosi tot predicatul *candidat*, dar cu un alt model de flux: (i,i).

- Acum avem de scris predicatul care generează soluțiile. Acesta va trebui să aibă următorii parametri:
 - o Lista din care luăm elemente – L
 - o Lista candidat în care adăugăm elemente unul câte unul - C
 - o Direcția (dacă suntem pe partea cu elemente descrescătoare sau elemente crescătoare)
 - o Rezultatul (doar în Prolog, nu și în model matematic)

generare(L, c₁...c_n, D)

$$\begin{aligned}
 &= 1. c_1 \dots c_n, \text{ dacă } D = 0 \\
 &2. \text{generare}(L, E \cup c_1 \dots c_n, 0), \text{ dacă } E = \text{candidat}(L), E \notin c_1 \dots c_n, E > c_1 \text{ și } D = 0 \\
 &3. \text{generare}(L, E \cup c_1 \dots c_n, 1), \text{ dacă } E = \text{candidat}(L), E < c_1 \text{ și } D = 1 \\
 &4. \text{generare}(L, E \cup c_1 \dots c_n, 0), \text{ dacă } E = \text{candidat}(L), E \notin c_1 \dots c_n, E > c_1 \text{ și } D = 1
 \end{aligned}$$

```

%generare(L:list, C:list, Directie:intreg, R:list)
%model de flux (i,i,i,o) nedeterminist
% L - lista inițială pentru care vom genera subșirurile având aspect de vale
% C - soluție candidat în care construim element cu element soluția
% Directie - 0, dacă suntem pe partea de descrescere, 1 dacă suntem pe
%           partea de creștere
% R - rezultat

```

```

generare(_, Cand, 0, Cand).
generare(L, [H|Cand], 0, R):-
    candidat(L, E),
    not(candidat([H|Cand], E)),
    E > H,
    generare(L, [E,H|Cand],0, R).
generare(L, [H|Cand], 1, R):-
    candidat(L, E),
    E < H,
    generare(L, [E,H|Cand], 1, R).
generare(L, [H|Cand], 1, R):-
    candidat(L, E),
    not(candidat([H|Cand], E)),
    E > H,
    generare(L, [E,H|Cand], 0, R).

```

- Ne mai trebuie un predicat care să apeleze predicatul *generare*. Din moment ce împărțim lista candidat în primul element și restul listei, nu putem începe candidatul cu lista vidă. Va trebui să generăm primul element (folosind predicatul *candidat*), și să apelăm predicatul *generare* având ca listă candidat lista cu acest element.
- Predicatul *generare* are o problemă: generează ca soluție și liste care au doar secvență descrescătoare. Deci, dacă tot va trebui să mai scriem un predicat pentru primul apel, în acest predicat vom genera primele 2 elemente din candidat, apelând *generare* doar dacă aceste 2 elemente sunt în ordinea potrivită.

$start(L) = generare(L, [E_1, E_2], 1)$, dacă $E_1 = candidat(L)$, $E_2 = candidat(L)$, $E_1 < E_2$

```
%start(L:list, Rez: list)
%model de flux (i,o) (i,i)
%L - lista inițială pentru care generăm subșiruri cu aspect de vale
%Rez - un subșir cu aspect de vale al șirului L
start(L, Rez):-
    candidat(L, E1),
    candidat(L, E2),
    E1 < E2,
    generare(L, [E1,E2], 1, Rez).
```

- Predicatul *start* generează subșirurile cu aspect de vale pe rând și putem să le vedem apăsând ; după fiecare soluție. Dacă vrem să avem o listă cu toate soluțiile (fără să trebuiască să apăsăm ;), putem folosi predicatul *findall* din Prolog.

$$startAll(l_1 l_2 \dots l_n) = \bigcup start(l_1 l_2 \dots l_n)$$

```
%startAll(L:list, Rez: list)
%model de flux (i,o) (i,i)
%L - lista inițială pentru care generăm subșirurile cu aspect de vale
%Rez - lista tuturor subșirurilor cu aspect de vale ale șirului L

startAll(L, Rez):-
    forall(R, start(L, R), Rez).
```

- Cum ar trebui modificată soluția noastră dacă dorim ca în soluție să avem elementele în aceeași ordine precum în lista originală?
- Din moment ce generăm candidatul în ordine inversă, după ce am adăugat un element în candidat vom putea adăuga doar elemente care în lista originală sunt în fața elementului adăugat. De exemplu din lista [5,4,3,6,7,-1,-2,-3], dacă am generat deja candidatul [6,7], următorul candidat trebuie generat doar din lista [5,4,3].
- Pentru a realiza acest lucru, cea mai simplă variantă este să modificăm predicatul candidat, pentru ca să returneze și această listă.

$$candidat2(l_1 \dots l_n, E, R) = \begin{matrix} 1. (l_1, \emptyset) \\ 2. (E, l_1 \cup R), \text{ unde } (E, R) = candidat2(l_2 \dots l_n) \end{matrix}$$

```
%candidat2(L:list, E:element, R:list)
%model de flux (i,o,o), (i,i,i), (i,o,i), (i,i,o)
%L - lista inițială
%E - un element din lista L
%R - sublista listei L până la elementul E
candidat2([E|_], E, []).
candidat2([H|T], E, [H|R]):-
    candidat2(T, E, R).
```

- candidat2([1,2,3,4,5], E, R) ne dă următoarele soluții
 - o E = 1, R = []
 - o E = 2, R = [1]
 - o E = 3, R = [1,2]

- $E = 4, R = [1, 2, 3]$
- $E = 5, R = [1, 2, 3, 4]$
- Dacă folosim predicatul `candidat2`, la predicatul `generare` apelul recursiv se face cu lista returnată de `candidat2`, nu cu lista originală `L`.